Diretoria

Martin Tygel *(Presidente)*
Clóvis Gonzaga *(1º Vice-Presidente)*
Luis Adauto Medeiros *(2º Vice-Presidente)*
Marco A. Raupp *(Secretário Geral)*
José Raymundo B. Coelho *(1º Secretário)*
Elisabeta Gallicchio *(2ª Secretária)*
Vera Lucia da Rocha Lopes *(Tesoureira)*

Conselho

José Luis Boldrini, Maria Cristina Cunha, Ricardo Kubrusly,
José Alberto Cuminato, João Frederico Meyer, Rubens Sampaio Filho,
José Mario Martinez Perez, Carlos Humes Júnior,
Dalcídio Moraes Cláudio, Sonia Gomes.

Comissão Organizadora

Alexandre M. Roma (USP)
Elisabeta Gallicchio (UFRGS)
Etzel Von Skert (UFSC)
José Raymundo B. Coelho (INPE)
Julio R. Claeyssen (UFRGS)
Luis Adauto Medeiros (UFRJ)
Marco Antônio Raupp (LNCC)
Marcos Arenales (USP-São Carlos)
Vanilde Bisognin (UFSM)

Secretaria da SBMAC

Rua Lauro Muller, 455
22290-160 Rio de Janeiro, RJ
Tel.: (021) 541-2132 Ramal 166

XX CONGRESSO NACIONAL DE MATEMÁTICA APLICADA E COMPUTACIONAL - CNMAC
Sociedade Brasileira de Matemática Aplicada e Computacional - SBMAC

INTRODUÇÃO AOS ALGORITMOS GENÉTICOS

Helio Jose Correia Barbosa
LNCC
Rio de Janeiro - RJ

GRAMADO - RS
1997

Apresentação

As presentes notas foram preparadas para servir de referência ao minicurso introdutório sobre Algoritmos Genéticos integrante do *XX Congresso Nacional de Matemática Aplicada e Computacional* realizado em Gramado, RS, de 8 a 12 de setembro de 1997.

Não há pre-requisitos para sua leitura e o que se buscou foi dar uma visão ampla da área sem entrar em muitos detalhes.

Sem a pretensão de originalidade, espero entretanto que o leitor descubra aqui algo de útil.

Desde já, aguardo e agradeço as correções e críticas que permitirão o aprimoramento do texto.

Rio de Janeiro, junho de 1997.

H.J.C.B.

Uma Introdução aos Algoritmos Genéticos

Helio J. C. Barbosa
Laboratório Nacional de Computação Científica/CNPq
e
Depto. de Ciência da Computação, UFJF
hcbm@lncc.br

1 Introdução

Uma parcela importante dos problemas tecno-científicos consiste em alguma forma de *busca*: dada uma certa coleção de elementos, deseja-se encontrar nela um ou mais elementos que atendam a algumas condições previamente especificadas. Um exemplo simples, mas de importância evidente, consiste em recuperar o conteúdo de um certo registro pertencente a um banco de dados da maneira mais rápida possível. Outro exemplo corresponde a encontrar uma trajetória que, partindo de um dado estado, permita alcançar um outro estado desejado da melhor maneira possível. Por vezes o problema apresenta tantas dificuldades que fica-se satisfeito com *alguma* solução que atenda a todas as restrições impostas.

Os processos de resolução dos problemas de busca dependerão fortemente do ambiente no qual esta busca se dá e das condições a serem satisfeitas pela solução. Entretanto, qualquer algoritmo de busca requer pelo menos

- uma forma de *representar* possíveis soluções para o problema e
- operadores capazes de *gerar* novas soluções candidatas

e envolve essencialmente a repetição dos passos

- gere solução candidata
- teste solução candidata

até que um critério de parada seja satisfeito.

Um algoritmo de busca é dito *cego* se a única informação disponível durante o processo é a resposta (sim ou não) à pergunta: é esta a solução procurada?

Se informações adicionais – dependentes do problema – podem ser obtidas durante a busca de maneira a avaliar a qualidade das soluções candidatas trata-se de um algoritmo ou busca dita *informada*.

De acordo com sua generalidade, os algoritmos de busca podem ser classificados como *fracos*, quando podem ser aplicados a uma gama variada de problemas, ou *fortes*, quando são projetados para aplicações mais específicas fazendo uso de uma quantidade maior de informações que são disponíveis para tal situação.

Como um exemplo simples tem-se o problema de encontrar uma raiz real da equação $f(x) = 0$ onde se supõe que a função f é contínua. Supondo também que a raiz desejada tenha sido isolada num intervalo (a, b), isto é, tem-se

$$f(a) \times f(b) < 0 \tag{1}$$

um algoritmo *fraco* seria o conhecido algoritmo da bisseção. Neste algoritmo, divide-se o intervalo (a, b) ao meio

$$x_0 = \frac{a+b}{2}$$

e testa-se o sinal de $f(x_0)$. O novo intervalo que contém a raiz tem x_0 como um extremo e a ou b como outro extremo, dependendo de qual deles corresponde a um valor de f de sinal contrário ao de $f(x_0)$. O processo se repete até que o intervalo que contém a raiz seja suficientemente pequeno. Tem-se assim um algoritmo extremamente fraco que pode encontrar a raiz da equação $f(x) = 0$ *sem o conhecimento da forma explícita de* $f(x)$, usando apenas o *sinal* que f assume num dado ponto x.

Um algoritmo mais forte pode ser definido reescrevendo-se a equação na forma $x = F(x)$ e fazendo as iterações

$$x_i = F(x_{i-1}) \quad i = 1, 2, \ldots \tag{2}$$

a partir de um certo $x_0 \in (a, b)$. Sob certas condições mostra-se que o processo pode convergir e, frequentemente, mais rapidamente do que no caso anterior. Naturalmente paga-se um preço por isso: $F(x)$ deve ser construída adequadamente para garantir a convergência do processo e isto requer o conhecimento explícito de $f(x)$.

Todos conhecemos um algoritmo mais forte ainda: é o método de Newton. Neste caso introduz-se mais conhecimento sobre a função $f(x)$: sua derivada em qualquer ponto. O algoritmo é também iterativo e se escreve como a Eq. (2) onde agora

$$F(x) = x - \frac{f(x)}{f'(x)}.$$

Sob certas condições mostra-se que a convergência agora pode ser *quadrática*! Mas, de novo, há um preço a ser pago por isto: $f(x)$ deve ser diferenciável para todo x, o

que reduz a aplicabilidade do método. E mais, por aparecer no denominador, $f'(x)$ não pode se anular para nenhum x_i. Pior ainda, na prática, $|f'(x)|$ não pode ser "pequeno" para evitar o risco de nos afastarmos de maneira irremediável da raiz procurada.

Resumindo, o primeiro método é mais fraco e, como um "jeep", é capaz, – sem desenvolver grande velocidade – de se locomover nas mais variadas situações, enquanto o último, o mais forte, requer, como um carro de corrida, condições adicionais para desenvolver altas velocidades.

Outro exemplo seria o problema comum de ajuste de curvas, isto é, de se encontrar uma função contínua $y = f(x)$ que melhor se ajuste a um conjunto de dados experimentais na forma $(x_i, y_i), i = 1, 2, \ldots, m$. Uma possível maneira de se medir a qualidade de tal ajuste é introduzir a quantidade

$$\left(\sum_{i=1}^{m} |y_i - f(x_i)|^p \right)^{1/p} \tag{3}$$

e procurar a função $f(x)$ que *minimize* esta grandeza. Formulado como está, o espaço de busca é o conjunto de todas as funções contínuas que é vasto e complexo: além de possuir dimensão infinita, não fornece indicações de como proceder a uma busca por um elemento – uma função $f(x)$ – que minimize a grandeza definida acima. Pode-se dizer tratar-se de uma situação de identificação *estrutural*.

Na prática é comum introduzir-se algumas restrições a tal espaço de busca a fim de tornar o problema tratável. Por exemplo, se a *forma* da função é especificada "a priori" e apenas alguns coeficientes $a_1, a_2, \ldots, a_n$ são deixados livres

$$f(x) = g(x; a_1, a_2, \ldots, a_n)$$

então o espaço de busca se reduz drasticamente ao dos vetores de n componentes; é o $\mathbb{R}^n$. Basta minimizar a função de n variáveis

$$\Psi(a_1, a_2, \ldots, a_n) = \left(\sum_{i=1}^{m} |y_i - g(x_i; a_1, a_2, \ldots, a_n)|^p \right)^{1/p}$$

utilizando-se um algoritmo de Programação Matemática.

Pode-se dizer tratar-se agora de um problema de identificação *paramétrica*, já que a *estrutura* (ou forma) da aproximação foi pré-definida de alguma maneira.

O caso $p = 2$ corresponde ao popular ajuste por *mínimos quadrados*. Uma condição necessária para que os a_i forneçam o ponto de mínimo de Ψ é

$$\frac{\partial \Psi}{\partial a_i} = 0 \quad i = 1, 2, \ldots, n$$

o que dá origem a um sistema de n equações não-lineares. Neste caso a busca se dará ainda em $\mathbb{R}^n$ mas podendo-se utilizar agora o algoritmo de Newton que explora o

conhecimento de que $\Psi(x)$ é *diferenciável* (o que não ocorre, por exemplo, se $p = 1$). Finalmente, se $f(x)$ tem a forma particular

$$f(x) = \sum_{i=1}^{m} a_i \phi_i(x)$$

o sistema obtido é *linear* e o problema se simplifica ainda mais.

Resumindo, fica claro que à medida que aumenta o conhecimento sobre o espaço de busca, especializam-se os algoritmos de busca para este espaço, com o aumento de eficiência e perda de generalidade, ou seja, pode-se passar de algoritmos mais fracos para algoritmos mais fortes.

Outra classificação importante é segundo a inclusão ou não de algum componente probabilístico no algoritmo. Quando alguma decisão é tomada a partir do valor de um número gerado (pseudo-)aleatóriamente diz-se que o algoritmo é *probabilístico* ou estocástico, e, em caso contrário, ele é dito *determinístico*. No primeiro caso, o gerador de números (pseudo-)aleatórios é sempre inicializado com algum valor (fornecido pelo usuário, por exemplo) e, assim, cada execução do algoritmo pode, em princípio, levar a um resultado diferente. À primeira vista, pode parecer estranho que um algoritmo com elementos probabilísticos possa ter alguma utilidade prática, acostumados que estamos com tantos algoritmos determinísticos eficientes em diversas áreas. Em muitas situações porém, vale a pena relaxar um pouco a atitude determinística e deixar algumas decisões ao acaso: um pouco menos de pretensão.

Para exemplificar duas características desejáveis – mas em geral conflitantes – num algoritmo de busca, considere o problema enfrentado por uma companhia de petróleo. Numa primeira fase ela deve ser capaz de identificar em diversas regiões – seu espaço de busca – aquelas áreas mais promissoras no que tange à existência de petróleo de qualidade e em quantidade. É a etapa de *exploração*. Mas isto não é tudo. Uma vez localizada uma região promissora é necessário, de maneira eficiente, retirar a maior quantidade possível de petróleo utilizando o mínimo possível de recursos. Esta etapa recebe na literatura inglesa o nome de *exploitation* aqui "traduzido" como *explotação*. Assim, um algoritmo de busca deverá ter também a melhor combinação possível de suas capacidades de *exploração* e *explotação*.

Em otimização, uma boa capacidade de exploração do espaço de busca é requerida se se deseja encontrar um extremo (máximo ou mínimo) *global*, ou, pelo menos, uma boa solução. Os algoritmos clássicos da Programação Matemática são projetados frequentemente para problemas *unimodais*, isto é, com apenas um extremo local (e portanto global), sendo assim essencialmente voltados ao aspecto da explotação. Quando aplicados a um problema *multimodal* sua convergência se dará para um extremo *local*, – raramente a solução global – que dependerá do ponto de partida utilizado.

Poli e Logan[Poli and Logan, 1996] identificam os ingredientes abaixo em algoritmos de busca

- uma *representação* das soluções candidatas
- um *critério de parada*
- um procedimento de *inicialização*
- uma *estrutura de memória* contendo as candidatas a serem operadas
- um procedimento de *gerenciamento da memória* que seleciona candidatas para inclusão/remoção da memória
- um procedimento de *seleção* da(s) candidata(s) a ser(em) operada(s)
- uma *estratégia de controle* da aplicação dos operadores
- um conjunto de *operadores* que agem sobre uma ou mais candidatas para gerar novas candidatas
- um procedimento de *reparo ou filtro* de soluções inválidas
- um procedimento de *avaliação da qualidade* de uma solução candidata
- um procedimento de *comparação da qualidade* de 2 candidatas
- um procedimento de determinação de um *gradiente de qualidade* que indique como melhorar uma solução candidata e
- uma *história* contendo a lista de soluções já testadas.

A enorme gama de variações possíveis nestes ingredientes ilustra os diferentes tipos de algoritmos já existentes e sugere novas possibilidades a serem testadas.

2 As origens da computação evolucionista

Hans-Paul Schwefel – ele mesmo um dos pioneiros da computação evolucionista [Schwefel, 1965] – considera difícil definir quem teria sido o primeiro a conceber um algoritmo evolucionista(AE). Em [Schwefel, 1994] ele cita alguns trabalhos pioneiros como os de Anderson(1953), Fraser(1957), Friedberg(1958) e Bremermann(1962) e ainda remete-nos ao seu livro[Schwefel, 1981b] onde mais referências desta fase inicial da computação evolucionista (CE) podem ser encontradas. Mais recentemente, pode-se detectar o desenvolvimento independente de três paradigmas da CE:

- As *estratégias evolucionistas* (EEs) (do alemão *evolutionsstrategie* surgiram a partir do trabalho de Ingo Rechenberg [Rechenberg, 1973], Hans-Paul Schwefel [Schwefel, 1965] e Peter Bienert em Berlin na década de sessenta.

  Inicialmente orientadas à otimização experimental, só mais tarde foram implementadas em computador e, desde o início, utilizavam variáveis contínuas e empregavam a mutação como o operador principal.

- Os algoritmos genéticos (AGs), inventados por John Holland no final da década de 60 visando o estudo de sistemas adaptativos complexos[Holland, 1975] mas logo aplicados a problemas de otimização na tese de De Jong[De Jong, 1975].

  Em contraste com as EEs, os AGs sempre utilizaram uma codificação binária e enfatizaram a recombinação (o *crossover*) como principal operador deixando a mutação com um papel secundário.

- A programação evolucionista (PE), introduzida por Lawrence Fogel[Fogel *et al.*, 1966] e mais difundida em [Fogel, 1992] e que visava, inicialmente, a construção de máquinas com número finito de estados para tarefas de previsão.

  A exemplo das EEs, a PE também utilizava a mutação como o operador básico.

Ultimamente vão ficando menos nítidas – e menos úteis – as fronteiras entre os três paradigmas apresentados uma vez que os algoritmos evoluem – via recombinação e mutação destas idéias iniciais – dando origem, por exemplo, a estratégias evolucionistas que usam operadores de recombinação e AGs que empregam codificação real.

Nestas notas apenas os AGs serão abordados.

3 O algoritmo genético

O algoritmo genético(AG) foi inventado por John Holland no final da década de 60 buscando inspiração no que se conhece sobre o processo de evolução natural, conhecimento este iniciado solidamente com a teoria da evolução de Darwin no seu famoso *The Origin of Species*.

Os interesses de John Holland entretanto não estavam limitados a problemas de otimização: ele estava[Holland, 1975] – e está[Holland, 1995] – interessado no estudo de sistemas adaptativos complexos, sejam eles biológicos (como o nosso sistema imunológico, por exemplo) ou não (como a economia global ou setorial, por exemplo).

Os AGs não são os únicos algoritmos inspirados na Biologia; as *redes neuronais* são baseadas no comportamento dos neurônios no cérebro e até o comportamento das formigas no processo de trazer alimento para o formigueiro já serviu de inspiração para a solução de problemas de otimização com variáveis discretas[Dorigo *et al.*, 1996] e contínuas[Bilchev and Parmee, 1995]. Além da Biologia, a Física também já inspirou algoritmos de otimização[Bounds, 1987].

Os AGs constituem uma classe de ferramentas muito versátil e robusta e que pode ser utilizada na solução de problemas de otimização embora não devam ser considerados estritamente minimizadores de funções[De Jong, 1992]. Quando usado como algoritmo de minimização um AG se distingue das técnicas mais comuns de programação matemática basicamente por

- empregar uma *população* de indivíduos, ou soluções
- trabalhar sobre uma *codificação* das possíveis soluções (genótipos) e não sobre as soluções (fenótipos) propriamente ditas
- empregar regras de transição *probabilísticas*
- não requerer informações adicionais (derivadas, por exemplo) sobre a função a otimizar

Assim, a busca de soluções pode se dar em conjuntos não-convexos[Hajela, 1990] e mesmo disjuntos, com funções objetivo também não-convexas e não-diferenciáveis podendo-se trabalhar simultaneamente com variáveis reais, lógicas e inteiras. É de se ressaltar também que os algoritmos genéticos não são facilmente presos a mínimos locais como é o caso dos algoritmos usuais de programação matemática. Quando utilizados no projeto, tais características dos AGs podem levar[Michielssen and Weile, 1995] à *inovação*: descoberta de soluções não convencionais que não poderiam ser vislumbradas pelo projetista por serem contra-intuitivas.

Um pseudo-código genérico capaz de englobar a maioria dos AGs existentes seria:

Algoritmo AG genérico
  Inicialize a população
  Avalie indivíduos na população
  Repita
    Selecione indivíduos para reprodução
    Aplique operadores de recombinação e mutação
    Avalie indivíduos na população
    Selecione indivíduos para sobreviver
  Até critério de parada satisfeito
Fim

onde os verbos utilizados, *inicialize, avalie, selecione e aplique* são, propositalmente, ainda insuficientes para definir um algoritmo computacional único.

Nas seções seguintes comenta-se um pouco sobre cada um dos procedimentos associados a tais verbos e, para ir fixando mais facilmente as idéias, considera-se

inicialmente o processo de reprodução num AG com população que será suposta de tamanho constante.

3.1 Reprodução

No que diz respeito à maneira pela qual os indivíduos criados são inseridos na população, considera-se a seguir dois tipos extremos de AGs.

Um extremo é o que pode ser chamado AG *generacional*, onde *toda* a população será substituída por novos indivíduos gerados pelo processo de seleção seguido da aplicação dos operadores genéticos:

Algoritmo AG generacional
Inicialize a população P aleatoriamente
Avalie indivíduos na população P
Repita
Repita
Selecione 2 ou mais indivíduos em P
Aplique operador de recombinação com probabilidade p_c
Aplique operador de mutação com taxa p_m
Insira novos indivíduos em P'
Até população P' completa
Avalie indivíduos na população P'
P $\leftarrow$ P'
Até critério de parada satisfeito
Fim

Como toda geração (de "pais") é integralmente substituída por outra mais nova (de "filhos") – não há "convivência" – corre-se o risco de se perder um bom indivíduo neste processo. Assim, especialmente em problemas de otimização, um procedimento frequentemente incluído neste algoritmo é o *elitismo*, ou seja, o(s) melhor(es) indivíduo(s) de uma geração é(são) preservado(s): uma cópia é passada diretamente à geração seguinte.

O outro extremo interessante de AG é aquele conhecido na literatura como "steady-state": apenas *um* indivíduo é criado de cada vez. Em seguida ele é avaliado e, de acordo com alguma estratégia, ele é inserido ou não na população, vindo a substituir algum outro indivíduo. A estratégia de inserção pode ser: o elemento é inserido se não for pior que todos os já existentes. A estratégia de substituição pode ser: substitua o pior de todos, por exemplo. Num algoritmo "steady-state" a população é ordenada de acordo com a qualidade de cada solução: o seu "ranking". Um pseudo-código seria:

Algoritmo AG "steady-state"
Inicialize a população P aleatoriamente
Avalie indivíduos na população P
Ordene a população P de acordo com a aptidão
Repita
Selecione operador genético
Selecione indivíduo(s) para reprodução
Aplique operador genético
Avalie indivíduo(s) gerado(s)
Selecione indivíduo f para sobreviver
Se f é melhor que o pior elemento de P **Então**
Remova um indivíduo da população
Insira f em P de acordo com seu "ranking"
Até critério de parada satisfeito
Fim

Fica claro que entre estes dois extremos pode-se imaginar AGs em que somente uma parte da população é substituída: "pais" e "filhos" podem vir a coexistir. Outros esquemas de reprodução podem também ser encontrados na literatura.

3.2 Representação e Codificação

A primeira etapa para se resolver um dado problema utilizando um AG consiste no estabelecimento de um esquema de codificação/representação para cada elemento do espaço de busca em questão já que os AGs não operam sobre tais elementos diretamente e sim sobre uma codificação dos mesmos. Utilizando a terminologia da Genética, cada elemento do espaço de busca é um *fenótipo* correspondente à realização de um dado código: o seu *genótipo*. Matematicamente, a escolha da codificação para um dado problema é a escolha da função ou regra que associa os elementos do espaço de genótipos com aqueles do espaço de busca, os fenótipos. Na prática os espaços de busca podem conter os mais variados entes matemáticos: vetores, matrizes, pesos e/ou conectividades de redes neuronais, expressões matemáticas, combinações de variáveis lógicas, inteiras e reais e mesmo *programas de computador* numa dada linguagem.

A forma mais comum de construção de uma codificacão corresponde a utilizar uma cadeia, de comprimento l, formada de caracteres tomados de um dado alfabeto. O caso mais comum é o binário, onde o alfabeto é composto dos símbolos 0 e 1. Assim, a cadeia 10010101 poderia representar uma possível solução de um dado problema. Neste caso, tem-se $l = 8$ e o conjunto dos genótipos é formado por todos os números binários de 00000000 a 11111111 contendo portanto $2^l = 2^8 = 256$

elementos. A codificação é a regra que associa a cada uma destas cadeias/números binários uma solução que pode ser uma variável real, inteira, etc. Concatenando-se o código de variáveis v_i de diferentes tipos pode-se ter uma solução codificada como uma cadeia

$$\underbrace{100101}_{v_1}\underbrace{1101}_{v_2}\cdots\underbrace{100111001}_{v_n}$$

também denominada *cromossomo*. Pode ser interessante associar mais de um cromossomo a cada indivíduo. O *Homo sapiens* é *diplóide*, isto é, possui 2 cromossomos. A grande maioria das aplicações de AGs utiliza entretanto indivíduos *haplóides*: com um só cromossomo. Resumindo, o genótipo é composto de um ou mais cromossomos que são compostos de *genes* construídos cada um por uma subcadeia de símbolos pertencentes ao alfabeto utilizado. Por exemplo, o ADN do *Homo sapiens* é representado utilizando os elementos do alfabeto $\{A, C, G, T\}$ que correspondem aos nucleotídeos adenina, citosina, guanina e timina, respectivamente.

Em muitas aplicações é suficiente o uso de uma representação *estática*, isto, é de "tamanho" fixo, seja ela um vetor, matriz, etc. Em outras isto não é suficiente e uma representação *dinâmica* deve ser adotada. Por exemplo, ao se utilizar um AG para evoluir um programa de computador para realizar certa tarefa[1] uma representação utilizada tem a estrutura de árvore cujo comprimento varia de um indivíduo para outro.

Fica claro neste ponto que a representação/codificação é uma etapa que, juntamente com a definição da função aptidão vista adiante, depende do problema a ser resolvido.

3.2.1 Um exemplo

Para exemplificar o efeito que a representação pode causar sobre os algoritmos utilizados para a solução de um dado problema, considere o caso simples de minimizar a função $f(x) = (x-8)^2$ no conjunto dos inteiros $I_{16} = \{0, 1, 2, \ldots, 15\}$. A solução exata do problema é, claramente, $x^* = 8$.

Denotando a vizinhança de um elemento x num dado conjunto de busca como $N(x)$ – entendendo que $x \in N(x)$ – pode-se definir o algoritmo geral de busca descrito na Figura1.

Trata-se de um algoritmo que simplesmente busca um mínimo *local* numa vizinhança N, isto é, ele pára quando

$$f(x^*) = \min_{x \in N(x^*)} f(x).$$

[1]Trata-se da chamada *Programação Genética*, um exemplo de AG muito especializado[Koza, 1992; 1994].

Algoritmo L
Inicialize : $k = 0$
Escolha um ponto (elemento) de partida x^k
Repita
Compute $f(x) \quad \forall x \in N(x^k)$
Denote por x^* aquele que fornece o menor valor para $f(x)$ em $N(x)$
Se $x^* = x^k$ **Então**
x^* é a solução
Pare
$x^k \leftarrow x^*$
$k \leftarrow k + 1$
Até critério de parada satisfeito
Fim

Figura 1: Algoritmo L

Para aplicar o algoritmo L ao nosso exemplo, considere inicialmente que o conjunto de busca seja exatamente I_{16} e que os inteiros estejam colocados em ordem crescente em forma de um anel onde o sucessor de quinze passa a ser o zero. Definindo o operador sucessor como

$$suc(k) = k + 1 \quad k = 0, 1, \ldots, 14 \quad \text{e} \quad suc(15) = 0$$

e o operador antecessor como

$$ant(k) = k - 1 \quad k = 1, 2, \ldots, 15 \quad \text{e} \quad ant(0) = 15$$

tem-se as propriedades

$$suc(suc(\cdots suc(x)))))))))))))))) = x \quad \forall x \in I_{16}$$

e

$$ant(ant(\cdots ant(x)))))))))))))))) = x \quad \forall x \in I_{16}$$

e pode-se definir a vizinhança de x como o conjunto

$$N(x) = \{ant(x), x, suc(x)\}.$$

É fácil ver que nesta representação o algoritmo L converge para a solução correta para *qualquer* ponto de partida: ele tem convergência global.

Considere agora que o mesmo conjunto onde a busca se dá, I_{16}, seja representado por dígitos binários. Dessa forma, ao zero associa-se a cadeia 0000, ao número um a

cadeia 0001, e assim por diante até o quinze, associado à cadeia 1111. Imagine agora que estas cadeias de dígitos binários estejam dispostas sobre um toro, isto é, cada elemento(número) tem um vizinho ao Norte, denotado $norte(x)$, ao Sul, $sul(x)$, à Leste, $leste(x)$ e à Oeste, $oeste(x)$. A característica destes 4 vizinhos de x é ter uma representação binária que difere em apenas 1 bit da representação de x.

Tem-se então as propriedades

$$norte(norte(norte(norte(x)))) = x \quad \forall x \in I_{16}$$

e

$$leste(leste(leste(leste(x)))) = x \quad \forall x \in I_{16}$$

e o mesmo valendo para os operadores $sul(\cdot)$ e $oeste(\cdot)$.

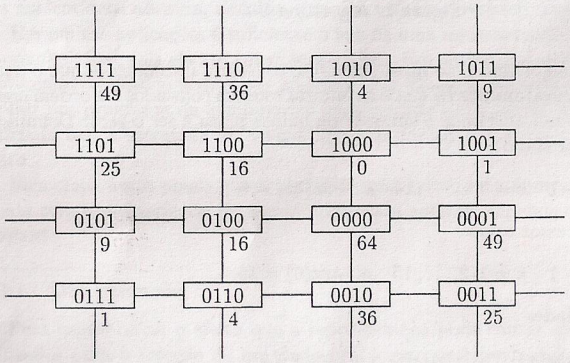

Figura 2: Representação binária e valores da função $f(x) = (x-8)^2$.

O Algoritmo L pode ser utilizado aqui também entendendo-se que os vizinhos de x são agora $norte(x), sul(x), leste(x)$ e $oeste(x)$ ou seja,

$$N(x) = \{x, norte(x), sul(x), leste(x), oeste(x)\}.$$

O que aconteceu com o problema ? Bem, a solução não se alterou: continua sendo $x = 8$ representada agora pela cadeia 1000. Mas pelo menos uma característica crucial para o nosso algoritmo se alterou: com a nova representação uma nova definição de vizinhança surgiu e o problema passou a ter *mais de um mínimo local* !

É fácil ver que o ponto $x = 7$, representado pela cadeia 0111 também é um mínimo local já que

$$f(norte(0111)) = 9, \quad f(sul(0111)) = 49,$$
$$f(leste(0111)) = 4, \quad f(oeste(0111)) = 25$$

e $f(0111) = 1$.

O que se pode dizer agora do Algoritmo L ? Existem *dois* mínimos locais para os quais o algoritmo pode convergir, mas apenas um deles é o mínimo global procurado. Surge então a noção de *bacia de atração* de um dado mínimo local: é o conjunto dos pontos de partida tais que o algoritmo converge para este mínimo local.

Pode-se verificar que a bacia de atração do mínimo global, para este algoritmo, é composta pelos pontos do conjunto B_G

$$B_G = \{1, 2, 8, 9, 10, 11, 12, 13\}$$

enquanto o algoritmo L converge para o mínimo local $x = 7$ (0111) sempre que partir de um ponto em B_L

$$B_L = \{3, 4, 5, 6, 7, 14, 15, 16\}.$$

Assim apenas *metade* dos pontos de partida nos levam à solução desejada!

Como os AGs frequentemente utilizam uma codificação binária é de se esperar então que o ocorrido neste exemplo simples ocorra realmente na prática: o conjunto de mínimos locais de uma dada função (de $\mathbb{R}$ em $\mathbb{R}$ em por exemplo), num espaço de Hamming difere daqueles encontrados no domínio original (em $\mathbb{R}$, por exemplo).

Uma prática nos AGs com codificação binária é utilizar os chamados códigos de Gray. De fato, a associação de cada inteiro com sua representação binária usual é arbitrária. Várias outras associações bi-unívocas podem ser imaginadas e utilizadas.

Voltando ao nosso exemplo, se no conjunto dos inteiros estamos no ponto $x = 7$, vê-se que a solução está próxima: ela dista de 1 unidade: é o vizinho $suc(7) = 8$. O mesmo não ocorre no espaço de Hamming das cadeias de 4 bits: o mínimo local $x = 7$ é representado por 0111 e o mínimo global $x = 8$ é representado por 1000. Na realidade 8 está na maior distância possível de 7. Todos os bits devem ser modificados para se passar de 7 a 8: a distância de Hamming é máxima: $d_H(0111, 1000) = 4$!

Este fenômeno, números naturais consecutivos que tem representação binária complementar (todos os bits devem ser trocados), recebe o nome na literatura de AGs de *Hamming cliffs*, ou seja, penhascos ou variações abruptas na distância de Hamming entre a representação binária de naturais consecutivos (de distância mínima, portanto).

Quando se utiliza o código de Gray usual, a associação entre binários e naturais no nosso problema fica como mostrado na Tabela 1.

Observa-se agora que cada cadeia difere de apenas 1 bit em relação à anterior, eliminando-se os *Hamming cliffs* citados acima.

Binário	Gray
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011
1110	1001
1111	1000

Tabela 1: Código de Gray.

3.2.2 Variáveis reais

Para representar uma variável real $x \in [x_{min}, x_{max}]$ o procedimento mais comum é utilizar l bits e associar a cadeia $\underbrace{00\ldots0}_{l\text{bits}}$ ao limite inferior x_{min} e a cadeia $\underbrace{11\ldots1}_{l\text{bits}}$ ao limite superior x_{max}. As demais cadeias intermediárias são mapeadas linearmente em pontos do intervalo $[x_{min}, x_{max}]$ igualmente espaçados de

$$\delta = \frac{x_{max} - x_{min}}{2^l - 1}.$$

Assim, se $x \in [0, 3]$ é codificada com 4 bits apenas tem-se, na realidade, x discretizada tal que

$$x \in \{0.0, 0.2, 0.4, \ldots, 2.8, 3.0\}.$$

Entretanto, pode ser conveniente introduzir uma relação não-linear entre os 2^l inteiros de $\underbrace{00\ldots0}_{l\text{bits}}$ a $\underbrace{11\ldots1}_{l\text{bits}}$ e os pontos de $x \in [x_{min}, x_{max}]$ de maneira que o espaçamento entre eles não mais seja constante.

3.3 A Inicialização

O processo de inicialização da população é quase sempre realizado aleatoriamente utilizando-se um gerador de números pseudo-aleatórios. No caso de codificação binária é fácil se obter uma boa distribuição de zeros e uns na população inicial, mas à medida que aumenta a cardinalidade do alfabeto utilizado na codificação isto vai se tornando mais difícil.

Pode-se também introduzir na população inicial um ou mais indivíduos "interessantes", como, por exemplo, soluções aproximadas conhecidas ou contendo algum tipo de informação prévia.

Cada execução de um AG pode ser replicada no computador bastando para isto fornecer a mesma "semente" para o algoritmo gerador de números pseudo-aleatórios. Um valor diferente da semente dá origem a uma outra sequência de números pseudo-aleatórios e, consequentemente, uma nova população inicial. Para dar uma idéia do comportamento de um AG numa dada aplicação, é comum computar-se estatísticas para um conjunto de resultados correspondentes a diferentes valores da semente.

3.4 A Função aptidão

A função aptidão é a responsável pelo processo de seleção dos indivíduos e deve indicar a qualidade de cada indivíduo/solução na população. Para o caso de otimização, ela está intimamente ligada à função objetivo que se deseja minimizar ou maximizar. No caso de maximização, por exemplo, da função objetivo $g(x)$, pode-se tomar a aptidão como

$$f(x) = g(x) + C$$

onde C é uma constante introduzida nos casos em que o processo de seleção exige que a aptidão seja positiva. À medida que a população evolui, os indivíduos passam a ter aptidão cada vez mais semelhantes entre si e pode ser necessário aumentar a *pressão de seleção* adotando-se algo como

$$f(x) = h(g(x))$$

com uma função h conveniente. Fica claro que não há necessidade de se incluir derivadas da função ojetivo e, consequentemente, um AG pode ser utilizado também – sem modificação alguma – para otimizar funções não-diferenciáveis.

Além disso, em outras aplicações a aptidão pode ter um carácter dinâmico, variando ao longo do processo evolutivo, estocástico, impreciso e mesmo *subjetivo*, no caso de algumas aplicações artísticas envolvendo pintura ou música, por exemplo.

A função aptidão deve ser tal que, através de sucessivas amostragens ao longo do processo evolutivo, forneça indicações sobre a localização do ótimo.

Por exemplo, uma função aptidão que retorna um valor unitário para a única solução do problema (como a combinação que abre um cofre) e zero para qualquer outra tentativa é incapaz de guiar a busca. A busca se torna completamente cega e todos os algoritmos são equivalentes a uma busca aleatória: encontrar a soluçãoé uma questão de sorte.

3.5 Esquemas de Seleção

Muitos esquemas de seleção já foram criados, analisados e implementados na prática de AGs (veja [Goldberg and Deb, 1991], por exemplo) mas, por simplicidade, apenas algumas dessas idéias serão aqui abordadas.

Inicialmente vale comentar que um esquema de seleção poderia ser enquadrado em uma das categorias[Kuo and Hwang, 1996]:

- *Seleção estabilizante*, também chamada normalizante, tende a eliminar indivíduos com valores extremos de aptidão,
- *Seleção direcional*, que tem o efeito de aumentar (ou diminuir, em caso de minimização) a aptidão média da população e é a usada na grande maioria dos AGs e finalmente a
- *Seleção perturbante*, que tende a eliminar os indivíduos com valores moderados de aptidão.

Nestas notas apenas esquemas direcionais serão considerados.

Uma das técnicas de seleção mais empregadas em AGs estabelece que a probabilidade p_i do i-ésimo indivíduo da população vir a ser selecionado para reprodução é proporcional à sua aptidão relativa. Uma possível implementação corresponde a tomar:

$$p_i = \frac{f_i}{\bar{f}}$$

onde $f_i = f(x_i)$ é assumida positiva e $\bar{f}$ é o valor médio da aptidão na população:

$$\bar{f} = \frac{\sum_{i=1}^{m} f_i}{m}.$$

Uma deficiência deste esquema se evidencia quando nas primeiras gerações surge uma solução medíocre mas tal que $f_i >> \bar{f}$. Esta solução pode dominar a população e acarretar uma *convergência prematura* para uma solução vizinha e ainda de baixa qualidade.

Outra deficiência surge ao final do processo quando toda a população pode ter valores de aptidão próximos de $\bar{f}$ acarretando

$$p_1 \cong p_2 \cong \cdots \cong p_n$$

que corresponde a uma pressão de seleção muito baixa e uma estagnação da busca.

Para minorar estas deficiências introduz-se algum tipo de tranformação na função aptidão[Goldberg, 1989].

Outra idéia é abandonar a *magnitude* da aptidão de cada elemento e sim utilizar somente a *posição relativa* (o "ranking") de cada indivíduo na população. Assim, mantendo a população ordenada por valores decrescentes da aptidão, a probabilidade de seleção de um indivíduo para a etapa de recombinação cresce com o seu "ranking": o primeiro do "ranking" tem a maior probabilidade de seleção.

Em certos tipos de aplicação uma terceira idéia é utilizada: o torneio. Num esquema de seleção via torneios, k elementos são sorteados aleatoriamente na população e o vencedor do torneio – aquele de maior aptidão entre os k indivíduos – é selecionado.

No caso do uso de AGs para o desenvolvimento de estratégias para um certo jogo com 2 participantes/adversários, é impraticável definir a aptidão de uma certa estratégia como o número de vitórias que ela obteria contra as possíveis estratégias. Usa-se então uma medida aproximada e relativa: a avaliação se dá via torneios: uma estratégia é posta em confronto com outra(s) e a vencedora do torneio é a selecionada.

3.6 Operadores Genéticos – codificação binária

Uma vez definida a representação dos elementos da população é possível construir operadores genéticos que, atuando sobre os genótipos, produzam novos indivíduos. Tais operadores são basicamente de dois tipos: recombinação e mutação.

3.6.1 Operadores de recombinação

Como o próprio nome indica, tais operadores visam promover a recombinação do material genético de (pelo menos) dois "pais" para a geração de um ou mais "filhos" que serão avaliados posteriormente. Tais operadores são também conhecidos na literatura inglesa como operadores de *crossover*, termo também oriundo da Genética.

CROSSOVER SIMPLES

Através do esquema de seleção implementado, dois indivíduos são escolhidos e, com uma probabilidade p_c, são submetidos à operação de recombinação. Uma

posição de "crossover" é sorteada e o material genético dos "pais" é recombinado conforme o esquema abaixo:

p_1 : 1111 111 f_1 : 1111 000

p_2 : 0000 000 f_2 : 0000 111

onde os "pais" p_1 e p_2 dão origem aos "filhos" f_1 e f_2 com a posição de "crossover" caindo, neste exemplo, entre a quarta e a quinta posições do cromossomo.

CROSSOVER DE DOIS PONTOS

Pode-se facilmente imaginar um operador onde dois pontos de "crossover" são sorteados e o material genético recombinado como no exemplo abaixo:

p_1 : 1111 111 11111 f_1 : 1111 000 11111

p_2 : 0000 000 00000 f_2 : 0000 111 00000

CROSSOVER UNIFORME

É fácil ver que, em princípio, o número de pontos de crossover poderia crescer. Na prática entretanto, este número raramente excede a dois. Um outro tipo de crossover também bastante utilizado é o chamado crossover uniforme onde todos os bits dos "pais" (e não apenas os contíguos) estão sujeitos a se "misturar" por ocasião da reprodução. Para a sua definição é conveniente introduzir o conceito de "máscara" para um crossover que permitirá também a descrição unificada de todos os outros tipos de crossover (1,2,... pontos).

Uma máscara de crossover é simplesmente uma cadeia de bits, de mesmo comprimento do cromossomo, que indicará a ocorrência ou não da troca de material genético naquela posição da cadeia. Assim, a máscara m_2 do crossover de 2 pontos exemplificado acima seria 000011100000 indicando as posições de troca:

p_1 : 1111 111 11111
p_2 : 0000 000 00000
m_2 : 0000 111 00000
f_1 : 1111 000 11111
f_2 : 0000 111 00000

No crossover uniforme[Syswerda, 1989] a máscara m é gerada aleatoriamente para cada ocorrência da operação de recombinação, estando o bit "1" com probabilidade 0.5 de ocorrência em cada posição.

OUTROS TIPOS DE RECOMBINAÇÃO

É fácil ver que um maior número de pontos de "crossover" pode ser utilizado. Além disso, pode-se recombinar o material genético de mais de dois "pais".

Quando outro tipo de codificação for utilizado pode ser interessante ou mesmo necessário introduzir outros tipos de operadores de recombinação; por exemplo, no caso em que as variáveis v_i do cromossomo forem representadas por valores reais.

3.6.2 Operadores de Mutação

Um ou mais operadores de mutação são normalmente introduzidos num AG. São operadores *unários* que, operando sobre um dado elemento da população, produzem uma modificação no seu genótipo dando origem a um fenótipo também modificado a ser avaliado posteriormente.

No caso de alfabeto binário, um operador de mutação fica facilmente definido pela modificação do símbolo/valor ocorrente em uma posição do cromossomo: se 1 ele passa a 0 e vice-versa. Este tipo de operação é efetuado de maneira que a probabilidade p_m de ocorrência de mutação de um dado gene é geralmente pequena; da ordem de $1/l$, por exemplo, onde l é o número de bits do cromossomo. Esta idéia também é oriunda da Genética e visa preservar alguma *diversidade* genética na população que vai perdendo-a à medida que a evolução avança e os indivíduos vão se tornando mais e mais semelhantes tendendo a uma solução que, nós casos de sucesso do AG, é uma boa solução para o problema. Além disso, a mutação proporciona uma forma de *exploração* do espaço de busca já que ela gera novos indivíduos, a serem avaliados, que podem revelar novas regiões promissoras no espaço de busca.

3.7 Operadores Genéticos – codificação real

Os AGs podem ser construídos utilizando-se uma codificação real para as variáveis, o que é mais natural quando as mesmas já são, por definição, contínuas. Assim sendo, o cromossomo passa a ser um vetor em $\mathbb{R}^n$ que coleciona as n variáveis reais que caracterizam a solução do problema.

Enquanto na codificação binária a precisão de cada variável é diretamente proporcional ao número de bits utilizados na sua representação, na codificação real a precisão é a própria precisão da máquina. Consequentemente, não haverá a necessidade dos processos de codificação e decodificação entre as representações binária e real.

Uma vantagem da codificação real é o fato de, em geral, ser mais intuitivo conceber operadores de recombinação e de mutação para um dado problema lançando mão de conhecimento já previamente adquirido no domínio da aplicação.

Nas seções seguintes usa-se a notação

cromossomo 1 $C_1 = (c_1^1, c_2^1, ..., c_n^1)$

cromossomo 2 $C_2 = (c_1^2, c_2^2, ..., c_n^2)$

para os cromossomos selecionados para as operações discutidas.

3.7.1 Operadores de recombinação

CROSSOVER DISCRETO

Este operador de recombinação simplesmente escolhe para cada variável um dos valores assumidos por esta variável nos cromossomos "paternos". Assim é gerado o cromossomo "filho" $F = (f_1, f_2, ..., f_n)$ onde cada f_i é igual a c_i^1 ou c_i^2 com igual probabilidade.

CROSSOVER UNIFORME

Neste caso o valor de cada componente f_i é escolhido de maneira aleatória (com probabilidade uniforme) entre os limites definidos pelos valores encontrados nos cromossomos "paternos", isto é

$$c_i^1 \leq f_i \leq c_i^2.$$

CROSSOVER DE 1 PONTO

Em analogia ao "crossover" de 1 ponto para a codificação binária pode-se definir um operador tal que produza os "filhos"

$$F_1 = (c_1^1, c_2^1, ..., c_{i+1}^2, ..., c_n^2) \quad \text{e} \quad F_2 = (c_1^2, c_2^2, ..., c_{i+1}^1, ..., c_n^1)$$

quando é sorteada a i-ésima posição para o corte.

CROSSOVER ARITMÉTICO

Neste caso os "filhos" $F_i = (f_1^i, f_2^i, ..., f_n^i)$ se definem via

$$f_i^1 = \lambda c_i^1 + (1-\lambda)c_i^2 \quad \text{e} \quad f_i^2 = \lambda c_i^2 + (1-\lambda)c_i^1$$

onde λ pode ser definido de várias formas.

CROSSOVER BLX_α

Gera o "filho" $F = (f_1, f_2, ..., f_n)$ com f_i escolhido, com probabilidade uniforme, no intervalo $[c_{min} - I \cdot \alpha, c_{max} + I \cdot \alpha]$ onde $c_{min} = min(c_i^1, c_1^2)$, $c_{max} = max(c_i^1, c_1^2)$ e $I = c_{max} - c_{min}$. Nota-se aqui que para $\alpha = 0$ recupera-se o "crossover" uniforme.

CROSSOVER HEURÍSTICO DE WRIGHT

Para este operador sorteia-se o real $r \in [0, 1]$ com probabilidade uniforme e, denotando por C_1 aquele de maior aptidão entre os cromossomos selecionados como "pais", define-se o cromossomo "filho" com componentes

$$f_i = r \cdot (c_i^1 - c_i^2) + c_i^1.$$

3.7.2 Operadores de mutação

No caso de codificação real, várias possibilidades se abrem para a construção de um operador de mutação específico para o problema em questão. Dois exemplos de operadores de mutação genéricos são

MUTAÇÃO ALEATÓRIA

Escolhe-se aleatoriamente um elemento no cromossomo, c_i, que tem seu valor alterado para algum $c_i^* \in [a_i, b_i]$ sorteado com probabilidade uniforme.

MUTAÇÃO NÃO-UNIFORME

Se este operador[Michalewicz and Schoenauer, 1996] é aplicado em uma certa geração gen e se o número máximo de gerações é $maxgen$, então:

$$c_i^* = \begin{cases} c_i + \Delta(gen, b_i - c_i) & \text{se } \tau = 0 \\ c_i - \Delta(gen, c_i - a_i) & \text{se } \tau = 1 \end{cases}$$

Na expressão anterior τ é escolhido aleatoriamente entre 0 ou 1 e a função $\Delta(gen, y)$ é dada por:

$$\Delta(gen, y) = y(1 - r^{(1-\frac{gen}{maxgen})^b})$$

onde $r \in [0, 1]$ é aleatório e o parâmetro b é definido pelo usuário, determinando o grau de dependência com o número de iterações.

3.8 Operadores Genéticos – problemas de permutação

Uma classe de problemas que apresenta características especiais quando resolvidos via AGs é aquela em que um alfabeto de cardinalidade $n > 2$ é utilizado para resolver problemas em que se busca uma permutação ótima de um certo conjunto de cardinalidade também n.

Um exemplo desta classe é o problema do caixeiro-viajante, de grande interesse prático e teórico. São dadas n cidades as quais o caixeiro-viajante deve visitar uma e só uma vez, retornando à cidade de partida de maneira a minimizar o custo total associado à rota percorrida.

Denotando as cidades por $0, 1, \ldots, n$ de maneira que a cidade número 0 seja a origem e destino final, vê-se que qualquer rota válida é uma permutação do conjunto $\{1, 2, \ldots, n\}$ e, assim, o conjunto onde a busca se dará tem cardinalidade $n!$ o que torna proibitiva a idéia de testar todas as possíveis permutações em busca daquela que minimiza o custo do percurso.

Para este problema de ordenação o cromossomo poderia ser simplesmente uma cadeia de n símbolos do alfabeto $\{1, 2, \ldots, n\}$.

Fica claro entretanto que elementos como $\{1, 3, 1, 4, 5 \ldots, n\}$ fazem parte do conjunto dos genótipos, que é

$$\{1, 2, \ldots, n\}^n,$$

sem no entanto constituirem um percurso válido (houve repetição da cidade 1). Mesmo que se construa a população inicial somente com rotas válidas (permutações) a utilização de um crossover usual para a codificação binária não garante que o indivíduo gerado seja de fato uma rota válida. Por exemplo, o crossover de 1 ponto para os cromossomos "pais"

123458967

132546789

gera indivíduos inválidos se o corte ocorrer na 2^a., 4^a., 6^a., 7^a., e 8^a. posições. É fácil ver que os outros tipos de crossover também geram elementos inválidos.

Se se insiste nesta representação, é conveniente então introduzir operadores genéticos especiais capazes de gerar não só soluções válidas como também que herdam características das soluções "pais" que as originaram.

Convém lembrar que problemas de ordenação como o do caixeiro-viajante podem apresentar restrições adicionais como, por exemplo,

- certa cidade deve ser visitada *antes* de uma outra ou
- certa cidade deve ser visitada antes de certa data

o que, embora reduza a cardinalidade do conjunto de busca, normalmente dificulta mais ainda a busca.

Vários operadores de recombinação já foram propostos para o problema do caixeiro-viajante, um deles sendo o

CROSSOVER PMX[2]

No crossover PMX, 2 pontos de corte são escolhidos aleatoriamente conforme ilustrado abaixo para $n = 10$

p_1	:	9	8	4	5	6	7	1	3	2	10
p_2	:	8	7	1	2	3	10	9	5	4	6

onde se define a correspondência entre os elementos do segmento interno e o estado intermediário

p_1	:	9	8	4	2	3	10	1	3	2	10
p_2	:	8	7	1	5	6	7	9	5	4	6

[2]Do inglês *partially matched crossover*.

Finalmente os segmentos externos dos cromossomos "filhos" são percorridos substituindo-se os elementos repetidos pelos seus correspondentes

p_1	:	9	8	4	2	3	10	1	6	5	7
p_2	:	8	10	1	5	6	7	9	2	4	3

4 Algoritmos Meméticos

É importante lembrar que uma possibilidade sempre presente nos AGs é a *hibridização*: pode-se sempre acoplar a ele um algoritmo já existente – especializado para o problema em questão – para efetuar uma busca local mais agressiva. Assim, pode-se pensar num operador de mutação generalizado, que, na realidade, efetue uma busca local, ou seja, tomando um indivíduo como ponto de partida, utilize um algoritmo específico para o problema em questão e efetue uma busca local, mais ou menos completa, adequada ao caso. Por exemplo, num problema de otimização em $\mathbb{R}^n$, um fenótipo pode ser usado como ponto de partida para um algoritmo de busca da Programação Matemática e o fenótipo/vetor melhorado pode ter sua aptidão aumentada.

Neste ponto duas opões se abrem: ou o genótipo mantém-se inalterado mas passa a ter a aptidão correspondente ao fenótipo melhorado ou o genótipo é modificado para incorporar a melhora descoberta no processo de busca fenotípica.

No primeiro caso tém-se o que parece ocorrer na Natureza: o melhoramento (aprendizagem) do fenótipo em seu desenvolvimento *não* altera o seu genótipo. Tudo deverá ser reaprendido pelos indivíduos que ele gerar. Mostra-se neste caso que mesmo sem passar seu aprendizado para sua prole (geneticamente) há uma vantagem evolutiva conhecida como efeito Baldwin que está, porém, além do escopo destas notas.

A opção de recodificação do genótipo corresponde ao que se denomina Lamarckianismo, devido ao naturalista frances Lamarck que acreditava serem herdadas pela prole certas características adquiridas em vida pelos pais. Embora aparentemente sem suporte biológico, o lamarckianismo algoritmico pode ser mais eficiente do que o AG tradicional explorando o feito Baldwin.

Deve-se notar que, num AG com codificação binária, se o fenótipo é, por exemplo, um vetor em $\mathbb{R}^n$, após a busca local pode não ser possível encontrar um genótipo correspondente a este vetor melhorado, visto que o espaço dos fenótipos possui neste caso muito mais elementos do que o espaço dos genótipos.

Um algoritmo de busca local geral para o caso de codificação binária corresponde a mudar todos os bits do genótipo – um por vez – decodificá-lo, avaliar a aptidão do vizinho assim gerado e substituir o genótipo por aquele que forneceu a melhor

aptidão. Para genótipos mais longos pode-se limitar o número de vizinhos (agora sorteados) para serem testados.

Outra possibilidade é sortear um bit, mudar seu valor, decodificar o genótipo, avaliar a aptidão do vizinho assim gerado e continuar o processo até encontrar o *primeiro* vizinho que apresente uma aptidão superior à inicial.

Ambos procedimentos são gerais no sentido de não dependerem do problema; efetuam modificações apenas sobre o genótipo sem necessidade de informações adicionais. Para alfabetos de cardinalidade maior do que a do binário, a etapa de mudança do valor do gene requer evidentemente outro sorteio para definir o vizinho a ser pesquisado.

O algoritmo resultante mantém a capacidade de exploração global do espaço de soluções – característica do AG inicial – e tem aumentada sua capacidade de busca local. Como os indivíduos da população agora são capazes de se aperfeiçoar via busca local – introduzindo uma forma de aprendizagem que modifica a evolução que se daria por via genética –, o algoritmo deixa de ser puramente genético e recebe denominações como AG híbrido ou ainda algoritmo memético.

5 Problemas com restrições

Em sua concepção original, os AGs trabalham supondo que a todo indivíduo pode ser associada uma aptidão. Como exemplo tem-se o problema de otimização em $\mathbb{R}^n$

$$\text{Maximize} \quad f(x_1, x_2, \ldots, x_N)$$

com restrições apenas do tipo

$$x_i^{(L)} \leq x_i \leq x_i^{(U)} \quad i = 1, 2, \ldots, n. \tag{4}$$

Pode-se tomar neste caso a função aptidão como a própria função $f(x_1, x_2, \ldots, x_N)$ que se quer maximizar, que é definida para qualquer vetor em $\mathbb{R}^n$ satisfazendo as restrições simples dadas por (4).

Isto nem sempre ocorre pois, por comodidade e frequentemente por necessidade, a representação utilizada é tal que o conjunto de todos os genótipos dá origem a um conjunto de fenótipos que contém elementos para os quais um valor de aptidão não está naturalmente definido.

O problema de otimização com restrições:

$$\begin{array}{lll} \text{Maximize} & f(x_1, x_2, \ldots, x_N) & \\ \text{sujeito a} & g_j(x) \geq 0 & j = 1, 2, \ldots, J \\ & h_k(x) = 0 & k = 1, 2, \ldots, K \\ & x_i^{(L)} \leq x_i \leq x_i^{(U)} & i = 1, 2, \ldots, N. \end{array}$$

serve de exemplo. Uma codificação binária ou real gera fenótipos (vetores em $\mathbb{R}^n$, no caso) que satisfazem às restrições do tipo (4). Não há como, em geral, satisfazer sempre às demais restrições.

Como advertência, vale notar que em muitas situações da engenharia as restrições não são sequer passíveis de serem colocadas numa forma explícita das variáveis x_i; são restrições implícitas cuja verificação de satisfação envolve tipicamente toda uma simulação computacional (via elementos finitos, para dar um exemplo).

Outro exemplo corresponde aos problemas de ordenação. Neste caso a solução é uma permutação dos inteiros de 1 a n. Se utilizamos um alfabeto de cardinalidade n nosso conjunto de genótipos contém elementos do tipo 23413... que não são admissíveis para o problema por conterem pelo menos um inteiro repetido (e pelo menos um ausente) não se constituindo portanto numa permutação.

Observe que não basta excluí-los da população inicial. Eles em geral podem reaparecer, dependendo dos operadores genéticos utilizados.

As 4 idéias mais encontradas na literatura para se aplicar um AG a um problema com restrições são: penalização, reparo, decodificadores especiais e operadores especiais e serão comentadas adiante.

Recentemente, AGs co-evolucionários[Potter and De Jong, 1994; Barbosa, 1996] – que mantem mais de uma população interagindo – foram aplicados na resolução de problemas com restrições mas seu estudo está em fase embrionária.

5.1 Penalização

A idéia mais simples quando se utiliza um esquema de penalização é adotar a *pena de morte*: um indivíduo não viável, por não ser possível avaliar sua aptidão ou por violar restrições, é simplesmente eliminado da população.

Embora de fácil implementação, pode estar sendo descartado material genético potencialmente útil. Existem situações – otimização em $\mathbb{R}^n$ com restrições, por exemplo – em que a manutenção destes indivíduos (ou uma fração deles, pelo menos) na população pode facilitar a obtenção de uma boa solução, uma vez que o ótimo pode ser aproximado tanto através de indivíduos não-viáveis, que dele se aproximam vindos do domínio infactível, quanto por soluções viáveis, oriundas do domínio factível.

O caso mais geral é tentar estimar a qualidade destas soluções infactíveis o que nem sempre é possível.

Em muitos casos isso pode ser feito através da introdução de uma função de penalização de maneira análoga ao que se faz em Otimização usual: as soluções que violam as restrições têm sua aptidão reduzida por este motivo. Mais ainda, pode ser conveniente penalizar mais aqueles indivíduos que violam um maior número de restrições e também fazer a penalização crescente com a magnitude da vio-

lação cometida. Por utilizarem gradientes, os algoritmos usuais de Programação Matemática requerem diferenciabilidade o que não ocorre aqui: pode-se utilizar funções de penalização discontínuas nos AGs.

Na maioria das implementações de penalização a escolha dos parâmetros para um esquema de penalização é dependente do problema e requer alguma experiência prévia.

5.2 Reparo

A idéia de reparo corresponde a introduzir operadores capazes de reparar, "consertar" os fenótipos inviáveis. Estes operadores são dependentes da aplicação e deverão ser construídos caso a caso. A aptidão do indivíduo pode ser reduzida de maneira crescente com o "esforço" gasto no processo de reparo.

5.3 Decodificadores Especiais

A introdução de decodificadores especiais pode ser muito útil mas também é dependente do problema. Suponha por exemplo um AG com codificação binária para um conjunto de n variáveis reais p_i que deverão ser não-negativas e satisfazer à restrição de igualdade

$$\sum_{i=1}^{n} c_i p_i = C \tag{5}$$

onde C e os c_i são constantes positivas conhecidas. Suponha ainda que a aptidão do indivíduo $\{p_1, p_2, \ldots, p_n\}$ é dada por $f(p_1, p_2, \ldots, p_n)$. O uso de um decodificador especial permite então a comodidade de considerar cada p_i pertencente ao intervalo $[0, 1]$ e fazer a busca no hipercubo $S = [0, 1]^n$, esquecendo-se, momentaneamente, da restrição (5).

Para avaliar a aptidão de um dado indivíduo $\{p_1, p_2, \ldots, p_n\}$ calcula-se inicialmente

$$v = \sum_{i=1}^{n} c_i p_i$$

e decodifica-se tal indivíduo como sendo

$$\{p'_1, p'_2, \ldots, p'_n\} = \{\frac{C}{v} p_1, \frac{C}{v} p_2, \ldots, \frac{C}{v} p_n\}$$

que agora satisfaz à restrição (5). A aptidão do indivíduo original é computada então como

$$f(\frac{C}{v} p_1, \frac{C}{v} p_2, \ldots, \frac{C}{v} p_n).$$

Neste caso, o decodificador especial utilizado pode ser interpretado como uma *projeção*: os indivíduos não-factíveis são projetados sobre o hiperplano definido pela Eq. (5) *antes* de terem sua aptidão avaliada. Uma aplicação pode ser vista em[Barbosa, 1997].

5.4 Operadores Especiais

A última idéia considerada aqui é partir de uma população inicial composta somente de elementos viáveis e, através de operadores genéticos especiais, *não gerar* indivíduos inviáveis.

Pode-se dizer que procura-se *fechar* o conjunto de busca X em relação às operações genéticas de recombinação e mutação. Denotando por g um operador genético que atua sobre n indivíduos e produza a prole denotada pelo conjunto P deve-se ter

$$g(X_1, X_2, \ldots, X_n) = P \subset X \quad \forall \{X_1, X_2, \ldots, X_n\} \subset X.$$

Com $n = 1$, g pode representar um operador de mutação.

Para exemplificar, considere a restrição

$$\sum_{i=1}^{n} x_i^2 = 1. \tag{6}$$

Para manter a viabilidade, e usando uma codificação real, Michalewicz e Schoenauer propoem um operador de recombinação especial[Michalewicz and Schoenauer, 1996]. Cada componente do indivíduo gerado é dada por

$$z_i = \sqrt{\alpha x_i^2 + (1 - \alpha) y_i^2}$$

onde x_i e y_i $(i = 1, 2, \ldots, n)$ são as componentes dos "pais" e $\alpha \in [0, 1]$ é escolhido aleatoriamente. É fácil ver que o indivíduo gerado sempre satisfaz à restrição (6).

Como operador de mutação eles propõem escolher aleatoriamente dois índices i e j diferentes e substituir x_i por px_i e x_j por qx_j onde

$$q = \sqrt{(\frac{x_i}{x_j})^2 (1 - p^2) + 1}$$

e $p \in [0, 1]$ é escolhido aleatoriamente.

Seguindo esta linha de raciocínio, operadores especiais podem ser construídos para outras restrições de estrutura semelhante.

6 Problemas com múltiplos objetivos

Na maioria das situações práticas, os projetistas formulam seus problemas como uma coleção de objetivos: grandezas a serem minimizadas ou maximizadas. Ocorre que tais objetivos são quase sempre conflitantes.

Por exemplo, o perfil de uma asa de avião deveria produzir o máximo de sustentação (a fim da maximizar a carga útil, entre outras coisas) provocando um mínimo de arrasto, visando minimizar o consumo de combustível.

Por falta de informação e de uma boa técnica de otimização multi-objetivo, subsequentemente os projetistas são forçados a simplificar artificialmente o problema original; por exemplo, introduzindo fatores de ponderação e combinando assim todos os objetivos numa única função.

Como os AGs trabalham com uma população de soluções, eles são naturalmente adequados para a determinação *simultânea* de *várias soluções* Pareto-ótimas, isto é, soluções *não-dominadas*, igualmente aceitáveis já que só é possível melhorar um objetivo piorando pelo menos um dos demais objetivos.

7 AGs Adaptativos e Auto-Adaptativos

Em sua forma mais simples, os AGs utilizam técnicas de seleção e geração independentes do domínio de aplicação e que ficam completamente definidas a partir do valor de parâmetros escolhidos pelo usuário e que não se modificam durante a análise. Por exemplo, na aplicação de um AG com codificação binária a um dado problema, pode-se optar por um "crossover" de 2 pontos utilizado com probabilidade $p_c = 0.75$ e uma mutação aplicada com uma taxa $p_m = 0.01$. Uma vez iniciada a análise, tais valores permanecem fixos, embora possa ocorrer que modificações convenientes durante o processo acarretassem numa busca mais eficiente.

Quando se dispõe de conhecimentos sobre o domínio de aplicação em questão pode ser possível – e geralmente vantajoso – incorporá-los através de representações e operadores genéticos mais apropriados.

Quando isto não puder ser feito, uma opção é o uso de AGs com variação dinâmica de parâmetros visando a aquisição e uso de informação sobre o cenário de aptidão em questão *durante* o processo evolutivo, com o objetivo de tornar a busca mais eficiente.

O caso mais simples de variação dinâmica de parâmetros corresponde a prescrever "a priori" e externamente uma certa lei de variação para um ou mais parâmetros. Por exemplo, pode-se definir que a probabilidade de aplicação do operador de "crossover", p_c, seja uma função decrescente do número de gerações. A justificativa para tal lei de variação é o fato de que, à medida que a população converge, vai perdendo diversidade e o "crossover" vai se tornando menos eficiente. Trata-se

de um AG com variação *exógena* de parâmetros: a mudança é pré-definida e não guarda, necessariamente, relação com o que de fato está ocorrendo com a população num dado problema.

Um AG recebe a denominação de *adaptativo* se a definição de parâmetros e operadores se dá de maneira *reativa* ao que está de fato ocorrendo durante o processo de busca num dado problema. Informações são colhidas ao longo do processo evolutivo e são usadas por *regras de atualização* dos parâmetros de controle.

A denominação AG *auto-adaptativo* é dada nos casos em que o próprio processo evolutivo da população de soluções do problema é usado para modificar (evoluir) os parâmetros de controle. Neste caso não há a necessidade de se introduzir esquemas de coleta e avaliação de informações ao longo do processo: parâmetros de controle são associados aos indivíduos e aqueles valores mais vantajosos conferirão aos respectivos indivíduos (e sua prole) vantagens competitivas que acarretarão numa proliferação dos valores vantajosos dos parâmetros de controle em detrimento dos valores prejudiciais ou de baixo desempenho como exemplificado mais adiante.

Em geral, o processo de adaptação pode se dar em vários níveis:

- nível populacional
- nível individual
- nível de componente

que serão sumarizados em seguida.

7.1 Adaptação ao nível populacional

Neste caso, parâmetros e definições globais do processo de busca são adaptados a partir de informações também globais, ou seja, envolvendo toda a população. Tipicamente, estatísticas da população e/ou do desempenho de operadores são utilizadas para tomar decisões que afetarão o processo de busca ao nível da população.

Davis[Davis, 1989] já utilizava um AG que adaptava as probabilidades de uso de cada um dos vários operadores de recombinação disponíveis de acordo com o desempenho de cada um deles na geração de indivíduos com boa aptidão. Isto requer a definição de esquemas de medição de tal desempenho bem como de atribuição de crédito a cada operador que pode envolver não só a prole imediata ("os filhos") como também mais gerações ("filhos" e "netos", por exemplo) [Julstrom, 1995].

Schraudolph e Belew[Schraudolph and Belew, 1992] propuseram que a própria codificação (binária) dos parâmetros de um AG fosse modificada durante a busca de maneira a aumentar a precisão dos valores encontrados.

Whitley et all[Whitley *et al.*, 1991] introduziram o "delta-coding" que, a partir de informações globais sobre a convergência do processo, muda a interpretação da

representação: uma variável passa a representar o afastamento em relação a um valor encontrado na busca e não mais o valor inicialmente procurado, com o objetivo de aumentar a precisão dos valores obtidos sem aumentar o número de bits utilizados na codificação.

Os exemplos listados até aqui correspondem a regras fixas de atualização de parâmetros. Spears[Spears, 1995] propõe um processo auto-adaptativo global para definir a probabilidade de aplicação do crossover uniforme em relação ao crossover de 1 ponto adicionando um bit ao cromossomo de cada indivíduo. Em cada ciclo reprodutivo, a percentagem de bits com valor 1 naquela posição ao longo da população define a probabilidade de aplicação do crossover uniforme. Assim as probabilidades de aplicação dos operadores vão variando automaticamente ao longo do processo evolutivo.

7.2 Adaptação ao nível individual

Além de não existir nenhuma razão para que todos os parâmetros de um algoritmo evolucionista sejam globais, não é muito difícil perceber que um dado indivíduo ocupando uma região mais "plana" do cenário de aptidão pode ser manipulado mais severamente do que aquele localizado numa crista.

Em [Schaffer and Morishima, 1987] é proposto um mecanismo de adaptação individual dos pontos de crossover. Cada indivíduo contém, após o código de uma solução, o código de uma máscara com a definição dos pontos de crossover. Quando dois indivíduos são selecionados para crossover, todos os pontos de crossover são utilizados e as indicações da máscara também são submetidas ao crossover, havendo assim tanto transferência de material genético quanto das instruções para crossover. O operador de mutação é aplicado a todo o cromossomo, inclusive a máscara.

A idéia é que aquelas máscaras de crossover que geram indivíduos mais bem sucedidos proliferam juntamente com eles ao longo do processo evolutivo, não sendo necessário introduzir nenhuma regra para adaptação da máscara: trata-se de um processo auto-adaptativo.

7.3 Adaptação ao nível de componente

Neste caso, parâmetros adaptativos são atribuídos a cada componente de uma dada solução, indicando como manipular tal componente independentemente das demais. Tais técnicas são mais comuns nas estratégias evolutivas para otimização em $\mathbb{R}^n$: cada variável v_i é calculada a partir do valor encontrado no cromossomo pai e utilizando parâmetros que se mostraram eficientes na tarefa de produzir melhores valores para aquele componente.

8 Morfogênese

Como já foi visto, uma característica fundamental dos algoritmos genéticos é a distinção entre o fenótipo, isto é, a solução candidata cujo desempenho é avaliado pela função aptidão, e o genótipo, que é a estrutura manipulada pelos operadores genéticos e que codifica tal fenótipo. Uma regra que associe a cada genótipo uma possível solução (fenótipo) é portanto requerida em cada aplicação.

Uma extensão que pode se mostrar vantajosa em várias aplicações é a interposição de mais um mapa ou regra entre genótipo e fenótipo: uma função de *desenvolvimento* ou *maturação* que associe a cada representação (genótipo) manipulada pelos operadores genéticos uma outra, em geral mais complexa, que será decodificada num fenótipo a ser avaliado pela função aptidão. Algoritmos evolucionistas com esta distinção entre a representação que é evoluída e a que irá ser decodificada e avaliada recebem a denominação de algoritmos evolucionistas *morfogenéticos*, ou *morfogênicos*[Angeline, 1995]. Tal termo é oriundo da Biologia, onde designa o processo de transformação do zigoto, recém formado por um crossover, num organismo completo cujo desempenho será testado em seu meio ambiente. O objetivo aqui entretanto é pragmático: não se trata de aumentar o realismo biológico por si só; trata-se de construir AEs mais eficientes.

Pode-se citar duas vantagens potenciais num AE morfogenético[Angeline, 1995]:

- Aumentar a *evolvabilidade* das soluções no sentido de reorganizar a representação visando facilitar a tarefa dos operadores genéticos em criar uma prole de desempenho superior e
- Comprimir/compactar a representação final (a ser avaliada) numa mais simples (que será manipulada), podendo-se adicionalmente reduzir o tempo de evolução de estruturas mais complexas.

É importante ressaltar o fato de que o espaço das estruturas mais complexas não estará, necessariamente, accessível à busca via a função de desenvolvimento utilizada.

Na prática, 3 tipos de função de desenvolvimento são encontradas:

- mapas quase triviais, de simples *tradução*, são os mais comuns e na maioria das vezes não chegam a criar uma estrutura muito mais complexa. O código de Gray é um exemplo desta classe onde o mapa é um-a-um e preserva o tamanho da representação. A vantagem que se observa é a eliminação dos *Hamming cliffs*, mencionados anteriormente nestas notas, com melhoria na evolvabilidade.

  Um exemplo mais complexo é o uso feito por Harp e Samad[Harp and Samad, 1991] de uma cadeia de caracteres binários (de comprimento variável) que é

decodificada numa rede neuronal completa incluindo parâmetros para o seu próprio treinamento.

- funções de *geração* capazes de construir uma estrutura complexa a partir de uma descrição compacta. Aplicações no desenvolvimento de arquiteturas de redes neuronais[Kitano, 1990], programação genética[Koza, 1992; 1994] e o uso dela para construção de redes neuronais[Gruau, 1992; 1993] bem como no desenvolvimento de criaturas artificiais[Sims, 1994] podem ser citadas.

- funções de desenvolvimento *adaptativas* são aquelas em que o mapeamento do genótipo para a representação a ser decodificada tem suas características modificadas dinamicamente ao longo do processo evolutivo.

  No trabalho já citado de Schraudolph e Belew[Schraudolph and Belew, 1992] uma adaptação dinâmica da codificação possibilita um aumento adaptativo da precisão das variáveis codificadas sem aumentar o tamanho da representação evoluída, o seu genótipo.

  Angeline e Pollack[Angeline and Pollack, 1994] mostram que a introdução de um operador de mutação que efetue a compressão de sub-árvores de um programa genético além de compactar a representação do programa evoluído, aumenta a sua evolvabilidade protegendo sub-árvores importantes de manipulação pelo operador de crossover. É interessante notar que este operador de compressão, como toda mutação, altera o genótipo, mas, ao contrário da maioria, não altera o fenótipo.

9 Algoritmos Genéticos Paralelos

A disponibilidade hoje de software e computadores com arquiteturas que permitem a computação paralela vem a aumentar as possibilidades de uso dos AGs. De fato, é fácil ver que a etapa de avaliação da aptidão de cada indivíduo da população é *naturalmente*[3] paralela. Uma possibilidade imediata é, portanto, introduzir o paralelismo nesta etapa, ou seja, separar as operações genéticas da avaliação da aptidão usando o paradigma mestre-escravos.

MESTRE-ESCRAVOS, SÍNCRONO

Neste AG, cada um dos processadores disponíveis – "escravos" – processaria um conjunto de indivíduos a ele atribuído enquanto o processador "mestre" efetua as demais operações. Embora "primitivo" e de implementação relativamente simples, este procedimento pode levar a ganhos substanciais no tempo total da análise visto que, em numerosas aplicações – especialmente em engenharia – o cálculo da aptidão

[3] Alguns diriam *embaraçosamente* paralela

pode ser muito caro, envolvendo, por exemplo, uma simulação completa de um modelo discretizado pelo método dos elementos finitos. Uma aplicação deste tipo é apresentada em[Guerreiro *et al.*, 1997], por exemplo.

A etapa de seleção requer, pelo menos em princípio, um esforço de *coordenação* ou *centralização*: os valores de aptidão de todos os indivíduos da população devem estar disponíveis simultaneamente no processador mestre para se proceder ao processo (global) de seleção. Isto introduz o necessidade de sincronismo dos processos.

Assim, os cromossomos de *todos* os indivíduos devem estar igualmente disponíveis a fim de se aplicar os operadores genéticos.

No caso em que

- a avaliação da aptidão requeira esforço computacional variado ou
- os processadores sejam heterogêneos ou ainda,
- submetidos a cargas variáveis,

este AG perde eficiência devido à necessidade do sincronismo que acarretará desbalanceamento de carga entre os processadores: haverá processadores ociosos aguardando que outros terminem suas respectivas tarefas. Uma maneira de minorar este problema é enviar um ou mais indivíduos para avaliação no processador escravo disponível no momento até que toda a população seja avaliada.

Naturalmente, o resultado em ambos os casos é o mesmo daquele encontrado pelo AG sequencial original.

Outra possibilidade é remover o sincronismo, dando origem a um novo AG:

MESTRE-ESCRAVOS, ASSÍNCRONO

Agora a seleção dos indivíduos é feita com a informação relativa à aptidão disponível no momento e a definição rigorosa do que seria uma geração se perde.

Fica claro que o resultado deste AG difere daquele obtido com sincronismo.

ASSÍNCRONO CONCORRENTE

Neste caso, n processadores idênticos fazem tanto operações de seleção, recombinação e mutação quanto de avaliação de aptidão sobre uma única população residente numa memória compartilhada por todos.

ILHAS

Outra idéia que surge é a subdivisão da população original de p indivíduos em n sub-populações, "residentes" nos n processadores disponíveis, submetida cada uma a um AG independente. Se não houver comunicação entre estes AGs tudo se passa como se o problema estivesse sendo resolvido com n AGs atuantes em n populações de tamanho p/n executados em paralelo o que não traz nada de novo. Entretanto, se se permite que indivíduos de uma sub-população *migrem* para outra sub-população então tem-se um novo AG.

Pode-se assim identificar um modelo de *ilhas*, onde periodicamente um ou mais indivíduos de uma sub-população migram para uma outra ilha/população *qualquer*, ou um modelo *trampolim*, onde esta migração se dá somente para *ilhas vizinhas*, bastante semelhante ao que ocorre em sistemas biológicos, onde a geografia limita a migração. Do ponto de vista computacional, esta geografia fica determinada com a definição de uma topologia de comunicação entre os n processadores que podem estar lógicamente conectados como um anel, toro, etc. através da definição dos vizinhos de cada processador/população.

Fica claro neste ponto a necessidade da introdução de parâmetros de controle adicionais para os AGs distribuidos: quando, quais e quantos elementos irão migrar, para onde irão, e como serão inseridos na nova população.

AGs CELULARES

Outra possibilidade corresponde ao caso de fazer o número n de sub-populações/processadores crescer até o limite interessante em que *cada* processador está associado a *um* indivíduo. Isto pressupõe um equipamento *maciçamente* paralelo com um software adequado.

De novo surge a necessidade de se introduzir uma vizinhança para cada processador/indivíduo e um esquema de seleção para a definição dos "pais" a serem utilizados na geração de novos indivíduos via recombinação/mutação. Além disso, um esquema de inserção destes novos indivíduos na população (provavelmente com substituição de algum indivíduo previamente existente) deverá ser implementado.

10 Co-evolução

Num AG padrão, a função aptidão não varia com o tempo, de maneira que pode se imaginar que a população explora e explota um cenário de aptidão fixo, gradualmente convergindo para um de seus picos: um máximo de aptidão.

Em contraste com estes cenários estáticos, a evolução biológica se dá num cenário de aptidão dinâmico, onde os organismos estão contínuamente se adaptando uns em relação aos outros e ao meio ambiente.

Por exemplo, se uma planta desenvolve (via uma entre tantas outras mutações que ocorrem em sua evolução) uma substância em sua seiva que provoca reações adversas no aparelho digestivo de um inseto que dela se alimenta, ela passa a ter uma vantagem sobre as demais e teria mais chances de se reproduzir e passar seus genes para suas descendentes.

Da mesma forma, um inseto que desenvolvesse (também por mutação) uma maneira de digerir aquele tipo de seiva passaria a ter uma vantagem sobre os demais e teria mais chances de se reproduzir e passar seus genes para seus descendentes.

Observa-se assim uma lenta, inconsciente e permanente "corrida armamentista" biológica.

A co-evolução de predador e presa e hospedeiro e parasita são particularmente interessantes.

A idéia de *co-evolução* já foi e está sendo aplicada na solução de problemas via AGs.

Potter e De Jong[Potter and De Jong, 1994] apresentaram um AG co-evolucionário *cooperativo* para otimização de funções. No caso em que n parametros devem ser encontrados, eles propõem o uso de n sub-populações, cada uma delas contendo candidatos para um dado parâmetro. A aptidão de um elemento de uma dada espécie/sub-população é medida de acordo com sua capacidade de cooperar com as outras espécies na construção de uma boa solução global. Estas idéias foram estendidas mais tarde para problemas mais complexos: redes neuronais[Potter and De Jong, 1995] e conjunto de regras[De Jong and Potter, 1995].

O trabalho pioneiro entretanto é o de Hillis[Hillis, 1990], onde a idéia explorada é a *competição*. O problema abordado por ele foi o da construção de redes para ordenação de números efetuando a menor quantidade possível de comparações. A aptidão de um ordenador é dada por sua habilidade em corretamente ordenar os conjuntos de números fornecidos como testes. Hillis observou melhores resultados com a co-evolução dos ordenadores e dos casos para testá-los. Assim, duas populações são mantidas: uma de ordenadores (hospedeiros) e outra composta de casos de teste (parasitas). Ambas as populações evoluiam conjuntamente interagindo através da função aptidão já que os parasitas tinham sua aptidão medida por sua capacidade em encontrar um erro nos ordenadores. Dessa forma, a aptidão de um indivíduo de uma população (ordenador ou caso de teste) depende dos elementos da outra população ! Fica claro que, mesmo introduzindo o elitismo, a função aptidão de um ordenador ou de um caso de teste não é monotonicamente crescente: tudo depende da evolução da outra população.

Mais recentemente, Paredis[Paredis, 1994] usou uma abordagem co-evolucionária para o projeto de redes neuronais via um AG. Duas populações são, de novo, mantidas: uma de redes neuronais e outra de exemplos da tarefa de classificação a ser submetida à rede neuronal em questão. A aptidão de uma rede era dada por sua capacidade de corretamante classificar os 20 últimos exemplos a ela submetidos enquanto a aptidão de um exemplo era o número de vezes que ele foi incorretamente classificado pelas últimas 20 redes que o analisaram. O processo de seleção fazia com que as melhores redes fossem mais frequentemente selecionadas para teste enquanto os exemplos de classificação mais difíceis também eram mais frequentemente apresentados às redes. Uma diferença em relação ao trabalho de Hillis é que, aqui, os exemplos eram sempre retirados de uma população fixa (200 casos pré-classificados) e não havia uma evolução desta população, no sentido do surgimento de novos exem-

plos.

Em [Barbosa, 1996], um AG co-evolucionário foi aplicado a problemas de min-max do tipo

$$\min_{x \in X} \max_{y \in Y} f(x, y)$$

onde x e y pertencem a duas populações independentes A e B que evoluem via AGs independentes, um de minimização e outro de maximização. A aptidão de um indivíduo $x \in X$ da população A é dada por

$$f_a(x) = \max_{y \in B} f(x, y)$$

enquanto a de indivíduo $y \in Y$ da população B é dada por

$$f_b(y) = \min_{x \in A} f(x, y).$$

O processo consistia em executar *max_gen_a* gerações no AG da população A e, em seguida, *max_gen_b* gerações no AG da população B. Este ciclo era repetido *max_cycles* vezes. Um pseudo-código seria:

Algoritmo
Inicialize população A
Inicialize população B
Para $k = 1, 2, ..., max_cycles$ **Faça**
Para $i = 1, 2, ..., max_gen_a$ **Faça**
gere nova população A
avalie nova população A
Para $j = 1, 2, ..., max_gen_b$ **Faça**
gere nova população B
avalie nova população B
Fim

Para um número total de gerações igual a

$$ng = max_cycles \times (max_gen_a + max_gen_b)$$

e para o caso de se fazer $max_gen_a = max_gen_b$, uma situação extrema seria tomar

$$max_gen_a = max_gen_b = 1 \quad \text{e} \quad max_cycles = ng/2$$

e a outra seria fazer

$$max_gen_a = max_gen_b = ng/2 \quad \text{e} \quad max_cycles = 1.$$

Esta última situação não apresenta atrativo algum visto que a população B só iniciaria sua evolução quando a população A já tivesse possivelmente convergido

em relação a um cenário de aptidão estático definido pela população inicial B. Intuitivamente, o melhor desempenho deve se dar para alguma faixa intermediária de valores para *max_gen_a* e *max_gen_b*. Um exemplo de aplicação[Barbosa, 1996] corresponde a encontrar o ponto de sela da função[Schwefel, 1981a]

$$f(x, y) = (1.5 - x(1 - y))^2 + (2.25 - x(1 - y^2))^2 + (2.625 - x(1 - y^3))^2$$

no domínio $S = [-5, 7] \times [-1, 2]$. A solução é o ponto $(0, 1)$ onde tem-se $f(x, y) \approx 14.20$. As execuções com o caso extremo $max_gen_a = max_gen_b = 1$ e $max_cycles = 50$ não apresentaram resultados confiáveis. Para os casos $max_gen_a = max_gen_b = 5$ e $max_cycles = 10$ e também $max_gen_a = max_gen_b = 10$ e $max_cycles = 5$ bons resultados foram obtidos e a Figura 3 mostra a média da aptidão do melhor elemento da população A ao longo do processo evolutivo tomada em 20 execuções.

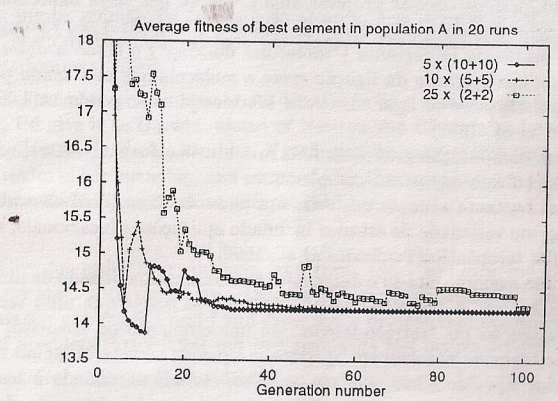

Figura 3: Média da aptidão do melhor elemento da população A em 20 execuções.

11 Algumas Aplicações

Qualquer tentativa de listar aqui todas as aplicações de AGs estaria fadada ao insucesso. Além de sua aplicação nos mais variados problemas de otimização em todas as áreas tecnológicas, os AGs têm sido aplicados também no estudo e modelagem de problemas científicos em geral. Por exemplo, os AGs podem ser usados na construção automática de programas para realização de tarefas específicas bem

como na construção de outras estruturas computacionais como autômatos celulares e redes neuronais. A área de robótica também utiliza AGs[Harvey *et al.*, 1992].

As áreas de economia e finanças tem recebido aplicações como gerenciamento de carteira de ações, desenvolvimento de estratégias para leilões, detecção de fraudes em movimentos na bolsa de valores[Moran, 1996], etc.

Os AGs começam a ser empregados também na modelagem e estudo de problemas em Ecologia, genética populacional, e na relação entre aprendizagem do indivíduo e evolução da espécie. Os sistemas imunológicos naturais são também estudados com o auxílio de AGs e, no sentido inverso, tem-se buscado nos sistemas imunológicos naturais inspiração para o projeto de sistemas de proteção de computadores contra a invasão de programas hostis, os infames "vírus".

Outra área de aplicação prática é o projeto de drogas na indústria farmacêutica. Como exemplo, pode-se desejar projetar uma droga tal que certa molécula se ligue fortemente a uma determinada enzima de maneira a impedir o seu funcionamento normal. Um problema importante ("molecular docking") é então a determinação da energia e da configuração da ligação entre a molécula que está sendo projetada e aquela à qual ela deve se ligar. Os AGs híbridos já estão sendo utilizados aqui também[Rosin *et al.*, 1997].

Também na computação gráfica os AGs foram utilizados[Sims, 1991] na criação de imagens, texturas e estruturas complexas.

Outra área bastante atuante dos AGs é a *Vida Artificial*[Mitchell and Forrest, 1994] com enorme variedade de estudos incluindo aplicações educacionais, artísticas e com potencial terapêutico[Pagliarini *et al.*, 1996].

Como último exemplo cita-se o desenvolvimento computadorizado de uma aplicação bastante prática: o "retrato-falado". Quando se deseja que uma testemunha ajude na construção do "retrato-falado" de um criminoso ou suspeito, a forma tradicional de trabalho consiste em apresentar tipos de caracteres faciais como por exemplo, vários tipos de olhos, queixos, cabelos, etc e ir solicitando à testemunha que indique aquele mais parecido com o da pessoa que foi vista por ela. Como frequentemente tal contato se deu por apenas alguns instantes, é mail fácil se lembrar do todo do que de detalhes isolados de uma face. Pesquisadores do departamento de Psicologia da New Mexico State University desenvolveram um sistema computadorizado para construção de retrato-falado que é baseado num algoritmo genético[Caldwell and Johnston, 1991]. Inicialmente a testemunha é apresentada a um conjunto de faces – gerado aleatóriamente – exibidas na tela de um computador. De acordo com o grau de semelhança de cada face com a face do suspeito, a testemunha então atribui notas a cada uma destas faces. Do ponto de vista de um AG, ela está definindo assim a aptidão de cada face/indivíduo da população. Um AG então promove a construção de uma nova população/geração de faces através de operadores de recombinação e mutação atuantes sobre o código/genótipo daquelas

faces/fenótipos selecionadas aleatoriamente mas de maneira a favorecer as de maior aptidão. De novo a testemunha atribui notas à cada face da nova geração, o AG é de novo utilizado e o processo se repete até que ocorra uma forma de convergência. O sistema funcionou bem em vários testes e seu autor, Victor Johnston, o patenteou.

Outras aplicações podem ser encontradas em[Goldberg, 1994] por exemplo e muitas mais vão surgindo dia a dia em variados ramos de atividade, tendo sido relatadas inclusive fora da literatura especializada[Naik, 1996; Port and Holyoke, 1995; Petzinger Jr., 1995; Williams, 1995; Moran, 1996]

12 Prazeres existenciais

O prazer encontrado por muitos daqueles que lidam com algoritmos genéticos deriva, na minha opinião, da beleza e do alcance, nem sempre devidamente percebido, da Teoria da Evolução iniciada por Darwin. De acordo com o filósofo Daniel C. Dennett[Dennett, 1996]

> If I were to give an award for the single best idea anyone has ever had, I'd give it to Darwin, ahead of Newton and Einstein and everyone else. In a single stroke, the idea of evolution by natural selection unifies the realm of life, meaning, and purpose with the realm of space and time, cause and effect, mechanism and physical law.

e ainda

> Give me order [Darwin] says, and time, and I will give you Design. Let me start with regularity – the mere purposeless, mindless, pointless regularity of physics – and I will show you a process that eventually will yield products that exhibit not just regularity but purposive design.

Para encerrar, me parecem adequadas as palavras de Lawrence Davis[Davis, 1991]:

> ... let me note that genetic algorithms, from their origin in Holland's research, have inspired passion in their adherents. Perhaps this is because they surprise us in interesting ways and because they exemplify processes we believe may have led to our own existence. Whatever the reason, there is something profoundly moving about linking a genetic algorithm to a difficult problem and returning later to find that the algorithm has evolved a solution that is better than the one a human found. With genetic algorithms *we* are not optimizing; we are creating conditions in which optimization occurs, as it may have occurred in the natural world. One feels a kind of resonance at such times that is uncommon and profound.

Referências

[Angeline and Pollack, 1994] P. J. Angeline and J. B. Pollack. Coevolving high-level representations. In C. Langton, editor, *Artificial Life III*, pages 55–71. Addison-Wesley, Reading MA, 1994.

[Angeline, 1995] P. J. Angeline. Morphogenic evolutionary computations: Introduction, issues and examples. In J. McDonnell, B. Reynolds, and D. Fogel, editors, *Fifth Annual Conference on Evolutionary Programming*. MIT Press, 1995.

[Barbosa, 1996] H.J.C. Barbosa. A genetic algorithm for min-max problems. In E.D. Goodman, V.L. Uskov, and W.F. Punch III, editors, *EvCA'96 - First Int. Conf. on Evolutionary Computation and its Applications*, pages 99–109, Moscow, 24–27 June 1996. Institute of High Performance Computer Systems of the Russian Academy of Sciences.

[Barbosa, 1997] H.J.C. Barbosa. A coevolutionary genetic algorithm for a game approach to structural optimization. In Th. Baeck, editor, *Proceedings of the Seventh International Conference on Genetic Algorithms and their Applications*, East Lansing, MI, July 1997.

[Bilchev and Parmee, 1995] G. Bilchev and I.C. Parmee. The ant colony metaphor for searching continuous design spaces. In T. Fogarty, editor, *Lecture Notes in Computer Science 993*, pages 25–39. Springer-Verlag, 1995.

[Bounds, 1987] D.G. Bounds. New optimization methods from physics and biology. *Nature*, 329:215–219, September 1987.

[Caldwell and Johnston, 1991] C. Caldwell and V.S. Johnston. Tracking a criminal suspect through face-space with a genetic algorithm. In Richard K. Belew and Lashon B. Booker, editors, *Proceedings of the Fourth International Conference on Genetic Algorithms and their Applications*. Morgan Kaufmann, San Mateo, CA, 1991.

[Davis, 1989] L. Davis. Adapting operator probabilities in genetic algorithms. In J.D. Schaffer, editor, *Proceedings of the Third International Conference on Genetic Algorithms and their Applications*, pages 61–69. Morgan Kaufmann San Mateo, CA, June 1989.

[Davis, 1991] L. Davis. *Handbook of Genetic Algorithms*. Van Nostrand Reinhold, New York, 1991.

[De Jong and Potter, 1995] K.A. De Jong and M.A. Potter. Evolving complex structures via cooperative coevolution. In *Fourth Annual Conference on Evolutionary Computation*, San Diego, CA, 1–3 March 1995.

[De Jong, 1975] K.A. De Jong. *Analysis of Behavior of a Class of Genetic Adaptive Systems.* PhD thesis, The University of Michigan, 1975.

[De Jong, 1992] K.A. De Jong. Genetic algorithms are NOT function optimizers. In *FOGA-92, Foundations of Genetic Algorithms*, Vail, Colorado, 24–29 July 1992.

[Dennett, 1996] D.C. Dennett. *Darwin's Dangerous Idea : Evolution and the Meanings of Life.* Touchstone Books, 1996.

[Dorigo *et al.*, 1996] M. Dorigo, V. Maniezzo, and A. Colorni. Ant system: Optimization by a colony of cooperating agents. *IEEE Trans. on Systems, Man and Cybernetics-Part B: Cybernetics*, 26(1):29–41, February 1996.

[Fogel *et al.*, 1966] L. J. Fogel, A. J. Owens, and M. J. Walsh. *Artificial intelligence through simulated evolution.* Wiley, New York, 1966.

[Fogel, 1992] D. Fogel. *Evolving artifical intelligence.* PhD thesis, University of California, San Diego, 1992.

[Goldberg and Deb, 1991] D.E. Goldberg and K. Deb. A comparative analysis of selection schemes used in genetic algorithms. In *Foundations of Genetic Algorithms, FOGA '91, proceedings*, 1991.

[Goldberg, 1989] D.E. Goldberg. *Genetic algorithms in search, optimization, and machine learning.* Addison-Wesley Reading, MA, 1989.

[Goldberg, 1994] D.E. Goldberg. Genetic and Evolutionary algorithms come of age. *Communications of the ACM*, 37(3):113–119, March 1994.

[Gruau, 1992] F. Gruau. Genetic synthesis of boolean neural networks with a cell rewriting developmental process. In J.D. Schaffer and D. Whitley, editors, *Combination of Genetic Algorithms and Neural Networks.* IEEE Computer Society Press, 1992.

[Gruau, 1993] F. Gruau. Genetic synthesis of modular neural networks. In S. Forrest, editor, *Fifth International Conference on Genetic Algorithms and its Applications.* Morgan Kaufmann San Mateo, CA, 1993.

[Guerreiro *et al.*, 1997] J.N.C. Guerreiro, H.J.C. Barbosa, E.L.M. Garcia, A.F.D. Loula, and S.M.C. Malta. Identification of reservoir heterogeneities using tracer breakthrough profiles and genetic algorithms. In *V LACPEC (Fifth Latin American and Caribean Petroleum Engineer Conference)*, Rio de Janeiro, August 1997.

[Hajela, 1990] P. Hajela. Genetic search – an approach to the nonconvex optimization problem. *AIAA Journal*, 28(7):1205–1210, July 1990.

[Harp and Samad, 1991] S. A. Harp and T. Samad. Genetic synthesis of neural network architecture. In *Handbook of Genetic Algorithms*, pages 202–221. Van Nostrand Reinhold, 1991.

[Harvey *et al.*, 1992] I. Harvey, P. Husbands, and D. Cliff. Issues in evolutionary robotics. Technical Report Cognitive Science Research Paper CSRP219, School of Cognitive and Computing Sciences, University of Sussex, Brighton BN1 9QH, England, UK, 1992.

[Hillis, 1990] W.D. Hillis. Co-evolving parasites improve simulated evolution as an optimization procedure. *Physica D*, 42:228–234, 1990.

[Holland, 1975] J. H. Holland. *Adaptation in Natural and Artificial Systems.* University of Michigan Press, Ann Arbor, 1975.

[Holland, 1995] J. H. Holland. *Hidden Order: How Adaptation builds Complexity.* Addison-Wesley Pub. Co., 1995.

[Julstrom, 1995] B.A. Julstrom. What have you done for me lately? Adapting operator probabilities in a steady-state genetic algorithm. In L.J. Eshelman, editor, *Proceedings of the Sixth International Conference on Genetic Algorithms and their Applications*, pages 81–87, Pittsburgh, PA, July 1995.

[Kitano, 1990] H. Kitano. Designing neural networks using genetic algorithms with graph generation system. *Complex Systems*, 4(4):461–476, August 1990.

[Koza, 1992] J. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection.* MIT Press Cambridge, MA, 1992.

[Koza, 1994] J. Koza. *Genetic Programming II: Automatic Discovery of Reusable Programs.* MIT Press Cambridge, MA, 1994.

[Kuo and Hwang, 1996] T. Kuo and S-Y Hwang. A genetic algorithm with disruptive selection. *IEEE Trans. on Systems, Man and Cybernetics-Part B: Cybernetics*, 26(2):299–307, April 1996.

[Michalewicz and Schoenauer, 1996] Z. Michalewicz and M. Schoenauer. Evolutionary algorithms for constrained parameter optimization problems. *Evolutionary Computation*, 4(1):1–32, 1996.

[Michielssen and Weile, 1995] E. Michielssen and D.S. Weile. Electromagnetic system design using genetic algorithms. In G. Winter, J. Périaux, M. Galán, and P. Cuesta, editors, *Genetic Algorithms in Engineering and Computer Science*, pages 345–369. John Wiley and Sons Ltd., 1995.

[Mitchell and Forrest, 1994] M. Mitchell and S. Forrest. Genetic algorithms and artificial life. *Artificial Life*, 1(3):267–289, 1994.

[Moran, 1996] N. Moran. Stock exchange gets fuzzy logic. *Financial Times*, September 4th. 1996.

[Naik, 1996] G. Naik. In sunlight and cells science seeks answers to high-tech puzzles. *Wall Street Journal*, January 16th. 1996.

[Pagliarini *et al.*, 1996] L. Pagliarini, H.H. Lund, O. Miglino, and D. Parisi. Artificial life: A new way to build educational and therapeutic games. In *Proceedings of Artificial Life V*. MIT Press/Bradford Books, 1996.

[Paredis, 1994] J. Paredis. Steps towards co-evolutionary classification neural networks. In R. Brooks and P. Maes, editors, *Artificial Life IV*. MIT Press/Bradford Books, 1994.

[Petzinger Jr., 1995] T. Petzinger Jr. At Deere they know a mad scientist may be a firm's biggest asset. *Wall Street Journal*, July 14th. 1995.

[Poli and Logan, 1996] Riccardo Poli and Brian Logan. On the relations between search and evolutionary algorithms. Technical Report CSRP-96-7, University of Birmingham, School of Computer Science, March 1996.

[Port and Holyoke, 1995] O. Port and L. Holyoke. Computers that think are almost here. *Business Week*, pages 48–52, July 17th 1995.

[Potter and De Jong, 1994] M.A. Potter and K.A. De Jong. A cooperative coevolutionary approach to function optimization. In *Third Parallel Problem Solving from Nature*, Jerusalem, Israel, 1994.

[Potter and De Jong, 1995] M.A. Potter and K.A. De Jong. Evolving neural networks with collaborative species. In *Proc. of the 1995 Summer Computer Simulation Conference*, Ottawa, Ontario, Canada, 24–26 July 1995.

[Rechenberg, 1973] Ingo Rechenberg. *Evolutiosstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann–Holzboog, Stuttgart, 1973.

[Rosin *et al.*, 1997] C. D. Rosin, R. S. Halliday, William E. Hart, and Richard K. Belew. A comparison of global and local search methods in drug docking. In *Proceedings of the Seventh International Conference on Genetic Algorithms and their Applications*. Morgan Kaufmann, San Mateo, CA, July 1997.

[Schaffer and Morishima, 1987] J.D. Schaffer and A. Morishima. An adaptive crossover distribution mechanism for genetic algorothms. In J. J. Grefenstette, editor, *Proceedings of the Second International Conference on Genetic Algorithms*, Hillsdale, New Jersey, 28-31 July 1987. Lawrence Erlbaum Associates.

[Schraudolph and Belew, 1992] N.N. Schraudolph and R.K. Belew. Dynamic parameter encoding for genetic algorithms. *Machine Learning*, 9(1):9–21, Jun 1992.

[Schwefel, 1965] Hans-Paul Schwefel. *Kybernetische Evolution als Strategie der experimentellen Forschung in der Strömungstechnik*. Dipl.-Ing. Thesis, Technical University of Berlin, Hermann Föttinger Institute for Hydrodynamics, March 1965.

[Schwefel, 1981a] H-P Schwefel. *Numerical optimization of computer models*. John Wiley & Sons, Chichester, 1981.

[Schwefel, 1981b] Hans-Paul Schwefel. *Numerical Optimization of Computer Models*. Wiley, Chichester, 1981.

[Schwefel, 1994] Hans-Paul Schwefel. On the evolution of evolutionary computation. In J. M. Zurada, R. J. Marks II, and C. J. Robinson, editors, *Computational Intelligence — Imitating Life*, pages 116–124. IEEE Press, Piscataway NJ, 1994.

[Sims, 1991] K. Sims. Artificial evolution for computer graphics. *Computer Graphics*, 25(4):319–328, July 1991.

[Sims, 1994] K. Sims. Evolving 3D morphology and behaviour by competition. In R. A. Brooks and P. Maes, editors, *Artificial Life IV*, pages 29–39. MIT Press Cambridge, MA, 1994.

[Spears, 1995] W. M. Spears. Adapting crossover in evolutionary algorithms. In J. McDonnell, R. Reynolds, and D. Fogel, editors, *Evolutionary Programming IV: Proceedings of the Fourth Annual Conference on Evolutionary Programming*, pages 367–384. MIT Press Cambridge, MA, 1995.

[Syswerda, 1989] G. Syswerda. Uniform crossover in genetic algorithms. In J.D. Schaffer, editor, *Proceedings of the Third International Conference on Genetic Algorithms and their Applications*, pages 2–9. Morgan Kaufmann, San Mateo, CA, 1989.

[Whitley *et al.*, 1991] L.D. Whitley, K. Mathias, and P. Fitzhorn. Delta-coding: an iterative search strategy for genetic algorithms. In Richard K. Belew and Lashon B. Booker, editors, *Proceedings of the Fourth International Conference on Genetic Algorithms and their Applications*, pages 77–84. Morgan Kaufmann, San Mateo, CA, 1991.

[Williams, 1995] T. Williams. Fuzzy, neural and genetic methods train to overcome complexity. *Computer Design*, May 1995.